Staj 15. Gün

15. Gün – 15. Bölüm: Docker Compose

1. Docker Compose Nedir ve Neden Var?

Buraya kadar Docker'da hep **tek bir container** üzerinde çalışıldı — Nginx, CentOS, Node.js container'ları birbirinden bağımsız açıldı. Ancak gerçek bir uygulama tek container'dan oluşmaz.

Örnek bir e-ticaret sistemi:

Kullanıcı → Nginx (Web Server) → Node.js (Backend) → MongoDB (Database) → Redis (Cache)
Sistem büyüdükçe her serviste birden fazla kopya çalışabilir:

Nginx
  │
Node.js ── Node.js ── Node.js
  │
MongoDB
  │
Redis ── Redis
Problem: Her container için ayrı ayrı şu adımlar gerekir → image oluşturma, container oluşturma, port verme, volume bağlama, network kurma, environment variable ekleme, başlatma, durdurma, silme. 15 container olduğunda bunu elle yönetmek pratikte imkansız hale gelir.

Tanım — Docker Compose: Birden fazla container'ı **tek bir YAML dosyasında** tanımlayıp, **tek komutla** oluşturmayı, başlatmayı, durdurmayı ve yönetmeyi sağlayan bir araçtır.

Compose Olmadan	Compose İle
`docker build`, `docker network create`, `docker volume create`, ayrı ayrı `docker run` komutları	Tek dosya: `docker-compose.yml`
Container sayısı arttıkça komut sayısı artar	Tek komut: `docker compose up`

Docker Compose sayesinde şunlar tek dosyadan yönetilir: container oluşturma, network oluşturma, volume oluşturma, image build etme, servisleri başlatma/durdurma/silme, servisler arası bağlantı kurma.

2. Servis (Service) Tanımı

Tanım — Servis: Docker Compose'da her container aslında bir **servis** olarak tanımlanır. Compose dosyasında yazılan her servis, çalıştırıldığında karşılığında bir container üretir:

1 Service → 1 Container

yaml
```yaml
services:
  web:      # → web container'ı
  backend:  # → backend container'ı
  mongodb:  # → mongodb container'ı
  redis:    # → redis container'ı
```
Servis bağımlılığı: Bazı servisler diğerleri hazır olmadan çalışamaz — örneğin backend, database açılmadan ayağa kalkamaz. Docker Compose bu sırayı `depends_on` ile yönetir:

1. MongoDB → 2. Redis → 3. Node.js (backend) → 4. Nginx (web)
Not: `depends_on` sadece **başlatma sırasını** garanti eder; servisin içindeki uygulamanın tam olarak hazır (örn. veritabanının bağlantı kabul ediyor) olduğunu garanti etmez.

3. Dockerfile ile Docker Compose Arasındaki Fark

	Dockerfile	Docker Compose
Tanım	Bir image'ın **nasıl oluşturulacağını** tarif eden dosya (Ubuntu kullan, Python kur, kodu kopyala, port aç,	Oluşturulan image'ları kullanarak **tüm sistemi** ayağa
Ürettiği	Image	Çalışan container'lar + network + volume
Kaps	Tek bir image	Birden fazla servis, birbirine

Dockerfile → Image → Docker Compose → Container'lar
→ Dockerfile **image üretir**, Docker Compose **sistemi çalıştırır**.

4. Docker Compose Kurulumu

İşletim sistemine göre değişir:

- **Windows / macOS** → Docker Desktop kurulduğunda Compose otomatik gelir, ekstra kurulum gerekmez.
- **Linux** → Docker kurulu olsa bile Compose her zaman hazır gelmeyebilir; ayrıca kurulması veya güncellenmesi gerekebilir.

Kurulu olup olmadığını kontrol etme:

bash
```bash
docker compose version
```
→ Sürüm bilgisi dönerse sistem hazırdır; hata dönerse Compose kurulu değil veya yanlış yapılandırılmış demektir.

Kullanılabilir komutları görme:

bash
```bash
docker compose --help
```
veya sadece:

bash
docker compose

5. docker-compose.yml Dosyası — Tanımı ve Amacı

Tanım: Docker Compose'un temelini oluşturan, sistemin tüm tanımlarını içeren **YAML** dosyasıdır. Uzantısı `.yml` veya `.yaml`; varsayılan ad `docker-compose.yml`, yeni sürümlerde `compose.yml`.

Bu dosyada şunlar tek yerde tanımlanır:

- Oluşturulacak container'lar (servisler)
- Kullanılacak image'lar veya build edilecek Dockerfile'lar
- Port yönlendirmeleri
- Volume'ler
- Network'ler
- Ortam değişkenleri (environment)
- Servis bağımlılıkları (depends_on)

Neden YAML? → İnsan tarafından kolay okunur/yazılır, konfigürasyon dosyaları için uygundur, Docker/Kubernetes ve birçok DevOps aracında standarttır.

Genel iskelet:

yaml
version: "3.9"

services:

networks:

volumes:
Dosya yukarıdan aşağı okunur.

`version` Alanının Tanımı (en çok karıştırılan konu)

version, Docker'ın sürümünü ifade ETMEZ. Asıl anlamı: docker-compose.yml dosyasının **hangi Compose dosya formatına (schema)** göre yazıldığını belirtir — yani Compose'a "bu dosyayı hangi kurallara göre yorumla" der.

Değer	Anlamı
`version: "2"`	Dosyayı Version 2 yazım kurallarına göre oku
`version: "3"`	Dosyayı Version 3 yazım kurallarına göre oku — Swarm desteği, deploy ayarları, replica yönetimi, CPU/RAM limitleri gibi ek özellikler içerir

→ Yeni Compose V2 sürümlerinde bu satır yazılmadan da kullanılabilir; format otomatik algılanır.

`services` Bölümünün Tanımı

Dosyanın **en önemli** kısmıdır — oluşturulacak bütün servisler burada tanımlanır. Her servis çalıştırıldığında bir container oluşturur.

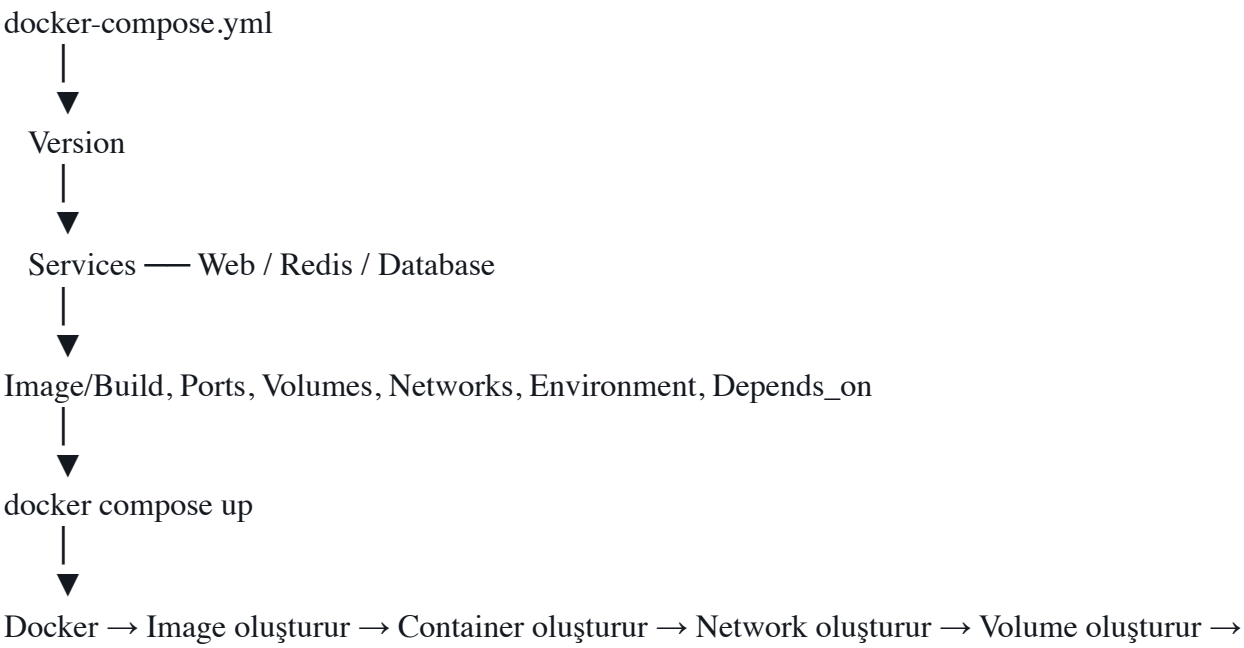
En sık kullanılan anahtarlar:

Anahtar	Görevi
`image`	Hazır bir image kullanır (örn. `nginx`, `mongo`)
`build`	Belirtilen dizindeki Dockerfile'dan image oluşturur
`ports`	Host ve container portlarını eşler (`"8080:80"`)
`volumes`	Kalıcı veri saklama için host/volume ile container'ı bağlar
`networks`	Servisin hangi network'e bağlı olacağını belirtir
`environment`	Container içine ortam değişkeni tanımlar
`depends_on`	Servisin hangi servisten sonra başlayacağını belirtir
`command`	Container başlarken çalıştırılacak komutu değiştirir

6. Docker Compose'un Çalışma Mantığı

`docker compose up` çalıştırıldığında Compose sırasıyla:

1. Gerekirse Dockerfile'lardan image oluşturur.
2. Hazır image'ları indirir.
3. Container'ları oluşturur.
4. Network'leri oluşturur.
5. Volume'leri oluşturur.
6. Servisler arası bağlantıları kurar.
7. Tüm uygulamayı çalıştırır.



7. Örnek Docker Compose Dosyası

Web + Backend + Database yapısı:

yaml

```yaml
version: "3.9"

services:

  web:
    image: nginx
    ports:
      - "8080:80"
    depends_on:
      - backend
    networks:
      - app-network

  backend:
    build: .
    ports:
      - "3000:3000"
    depends_on:
      - database
    networks:
      - app-network

  database:
    image: mongo
    volumes:
      - db-data:/data/db
    restart: always
    environment:
      MONGO_INITDB_ROOT_USERNAME: admin
      MONGO_INITDB_ROOT_PASSWORD: 123456
    networks:
      - app-network

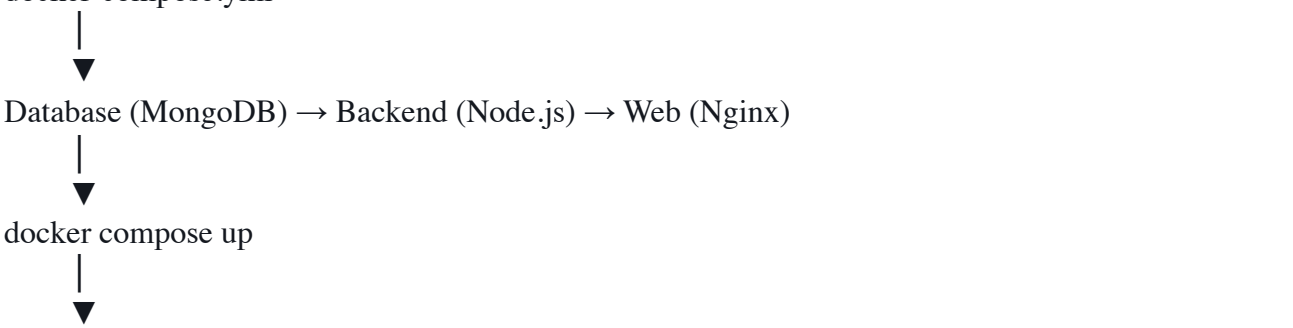
volumes:
  db-data:

networks:
  app-network:
    driver: bridge
```

Bu dosya ne yapıyor?

- **database** → `mongo` image'ı indirilir, `db-data` volume'ü veri kaybını önlemek için bağlanır, kullanıcı/şifre `environment` ile ayarlanır, `restart: always` sayesinde container düşerse otomatik yeniden başlar.

- **backend** → hazır image yerine `build: .` ile bulunduğu dizindeki Dockerfile çalıştırılarak image oluşturulur, 3000 portu açılır, `depends_on: database` sayesinde database'den sonra başlar.
- **web** → hazır `nginx` image'ı kullanılır, host'un 8080 portu container'ın 80 portuna yönlendirilir, `depends_on: backend` ile backend'den sonra başlar.
- Üçü de aynı **app-network**'e bağlanır → böylece birbirlerine **servis adıyla** (örn. `database`) erişebilirler.



8. Docker Compose Komutları (Detaylı)

Komutların çalışması için bulunduğun dizinde bir `docker-compose.yml` (veya `compose.yml`) dosyası olmalıdır.

Genel komut yapısı:

bash
docker compose [opsiyon] <komut> [parametre]
docker compose <komut> --help

`docker compose up`

Tanım: Compose dosyasındaki servisleri **oluşturur ve çalıştırır**. Dosya okunur, gerekiyorsa image build edilir, container'lar oluşturulur (create), servisler başlatılır (start). Daha önce oluşturulmuş servisler yeniden oluşturulmaz, sadece başlatılır.

bash
docker compose up

Opsiyon	Görevi
`docker compose up -d`	Servisleri arka planda (detached mode) çalıştırır
`docker compose -f compose-dev.yml up`	Varsayılan yerine belirtilen dosyayı kullanır
`docker compose up --no-recreate`	Mevcut container'lara dokunmadan sadece eksik servisleri oluşturur
`docker compose up --force-`	Değişiklik olmasa bile tüm container'ları yeniden
`docker compose up --remove-`	Dosyada artık bulunmayan eski servisleri kaldırır

`docker compose down`

Tanım: Servisleri **durdurur** ve Compose tarafından oluşturulan container'lar ile network'leri **siler**. Varsayılan olarak volume ve image'lara dokunmaz.

bash
docker compose down

Opsiyon	Görevi
`docker compose down -v`	Compose'un oluşturduğu volume'leri de siler (veri kaybı riski!)
`docker compose down --rmi`	Compose'un kullandığı tüm image'ları siler
`docker compose down --rmi local`	Sadece kullanılmayan (dangling/local) image'ları siler

`docker compose ps`

Tanım: Compose tarafından oluşturulan servisleri listeler — çalışan servisler, port bilgileri ve container durumları görüntülenir.

bash
docker compose ps

`docker compose exec`

Tanım: Çalışan bir container içinde komut çalıştırmak için kullanılır.

bash
docker compose exec <servis> <komut>
docker compose exec web bash

Opsiyon	Görevi
`-d`	Komutu arka planda çalıştırır
`-u`	Belirtilen kullanıcı ile çalıştırır

`docker compose build`

Tanım: Compose dosyasında `build` tanımlanan servisler için Dockerfile'dan image oluşturur. **Sadece image üretir, container oluşturmaz.**

bash
docker compose build

`docker compose stop`

Tanım: Çalışan servisleri **durdurur ama silmez** (container diskte kalır).

bash
docker compose stop
docker compose stop web # sadece web servisini durdurur

`docker compose start`

Tanım: Durdurulmuş servisleri **tekrar başlatır** (container'ı yeniden oluşturmaz, var olanı ayağa kaldırır).

bash
docker compose start
docker compose start web

`docker compose run`

Tanım: Belirtilen servis üzerinde **tek seferlik** bir komut çalıştırır; gerekirse bağımlı servisleri otomatik başlatır. Genelde geçici, tek kullanımlık işlemler için kullanılır (örn. bir migration script'i çalıştırmak).

bash
docker compose run <servis> <komut>

`docker compose rm`

Tanım: Durdurulmuş container'ları siler.

bash
docker compose rm

Opsiyon	Görevi
`-f`	Zorla siler (onay istemez)
`-s`	Önce container'ı durdurur, ardından siler

`docker compose logs`

Tanım: Container'ların ürettiği logları görüntüler.

bash
docker compose logs
docker compose logs web # sadece web servisinin logları

Özet Tablo — Tüm Komutlar

Komut	Görevi
`docker compose up`	Servisleri oluşturur ve başlatır
`docker compose up -d`	Servisleri arka planda çalıştırır

`docker compose down`	Container ve network'leri durdurup siler
`docker compose down -v`	Volume'leri de siler
`docker compose ps`	Çalışan servisleri listeler
`docker compose exec`	Çalışan container içinde komut çalıştırır
`docker compose build`	Dockerfile'dan image oluşturur
`docker compose stop`	Servisleri durdurur
`docker compose start`	Durdurulan servisleri yeniden başlatır
`docker compose run`	Servis üzerinde tek seferlik komut çalıştırır
`docker compose rm`	Durdurulmuş container'ları siler
`docker compose logs`	Servis loglarını görüntüler

Ezber (en çok kullanılan 5 komut):

bash
```
docker compose up -d        # Kur ve arka planda çalıştır
docker compose ps           # Duruma bak
docker compose logs sirket1   # Hataları incele
docker compose stop         # Geçici durdur
docker compose down         # Tamamen kaldır
```
→ up: kur ve çalıştır · ps: duruma bak · logs: hataları incele · stop: geçici durdur · down: tamamen kaldır

9. Genel Akış (Uçtan Uca Özet)

```
Uygulamayı hazırla
  → Dockerfile oluştur (gerekiyorsa)
    → docker-compose.yml yaz
        ├── Servisleri tanımla
        ├── Image veya Build belirt
        ├── Portları eşleştir
        ├── Volume ekle
        ├── Network oluştur
        ├── Environment değişkenlerini tanımla
        └── depends_on ile bağımlılıkları belirt
    → docker compose up
        → Docker: network oluşturur, volume oluşturur, gerekirse image build eder,
                container'ları oluşturur, servisleri başlatır
    → Uygulamayı tarayıcıdan test et
    → docker compose ps / logs / exec ile kontrol et
    → İş bitince: docker compose down (veya down -v)
```

10. Örnek Senaryolar (Kapsam)

- **Örnek 1** → 2 adet Nginx servisi, network ve volume kullanımı.
- **Örnek 2** → Node.js + MySQL, `build` ile Dockerfile'dan image, `depends_on` kullanımı.
- **Örnek 3** → WordPress + MySQL + phpMyAdmin, gerçek bir web uygulamasının ayağa kaldırılması.
- **Örnek 4** → Drupal + aynı MySQL, mevcut Compose dosyasına yeni servis eklenmesi, aynı veritabanının birden fazla uygulama tarafından kullanılması.

Kritik noktalar:

- Aynı network'teki servisler birbirine **servis adıyla** erişir (örn. `db`).
- `depends_on` yalnızca başlama **sırasını** belirler, uygulamanın içeriden hazır olduğunu garanti etmez.
- `down -v` volume'leri de sildiği için **veri kaybına** yol açabilir.

16. Bölüm: Docker Swarm

1. Docker Swarm Nedir?

Docker Compose, **tek bir makinedeki** çok sayıda container'ı yönetmeyi sağlıyordu. Ancak uygulama büyüdüğünde tek sunucu yetersiz kalır, yeni sunucular eklenir ve container sayısı yüzlerce hatta binlerce olabilir. Bu ölçekte sunucuları ve container'ları tek tek yönetmek pratikte imkansız hale gelir.

Tanım — Docker Swarm: Birden fazla Docker host'unu **tek bir küme (cluster)** gibi yöneten, servisleri dağıtan, ölçeklendiren ve kesintisiz çalışmasını sağlayan Docker'ın **yerleşik (built-in) orkestrasyon aracıdır**.

2. Docker Swarm Neden Gereklidir?

Tek host üzerinde çalışırken:

Host 1
├── Container
├── Container
├── Container
└── Container

Kaynaklar tükendiğinde yeni host'lar eklenir:

Host 1 Host 2 Host 3

Ama bu noktada yeni sorular ortaya çıkar:

- Hangi container hangi host'ta çalışıyor?
- Yeni container nereye kurulacak?
- Bir host çökerse ne olacak?

- Trafik nasıl dağıtılacak?
- Servis kesintisiz nasıl devam edecek?

→ Bunları yönetecek bir sisteme ihtiyaç doğar; bu ihtiyacı Swarm karşılar.

3. Orkestra Benzetmesi

Bir orkestrada keman, davul, trompet, piyano tek başına iyi çalabilir — ama bir **şef** olmadan herkes farklı zamanda çalar ve ortaya uyumsuz bir müzik çıkar. Şef geldiğinde kimin ne zaman çalacağını, kimin duracağını, kimin başlayacağını yönetir.

→ Container'lar orkestradaki müzisyenlerdir, **Swarm ise orkestra şefidir.**

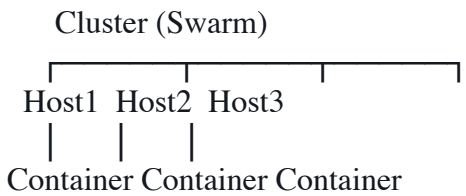
4. Docker Swarm'ın Görevleri (Tanım Detayı)

Swarm şunları yapar:

- Container'ları yönetir.
- Servisleri host'lara dağıtır.
- Gerekirse yeni container oluşturur.
- Yükü host'lar arasında dengeli dağıtır (**Load Balancing**).
- Arızalanan host'taki servisleri başka host'ta tekrar ayağa kaldırır.
- Servislerin sürekli çalışmasını sağlar (**High Availability**).
- Cluster yönetimini gerçekleştirir.

5. Cluster Tanımı

Tanım — Cluster: Birden fazla sunucunun **tek bir sistem gibi** çalışmasıdır.



Swarm bu sunucuları tek bir havuz gibi görür → uygulamanın Host1'de mi, Host2'de mi yoksa Host3'te mi çalıştığını bilmen gerekmez, buna Swarm kendisi karar verir.

6. Host Çökerse Ne Olur?

Normal Docker'da (Single Mode):

Host1 → Container A / B / C
Host çökerse → Container'lar da kapanır → Servis kesilir
Swarm'da:

Host1 ❌
|
▼
Swarm
|
▼
Host2 → Container A (yeniden ayağa kaldırılır)
Host3 → Container B, C (zaten çalışıyor)
→ Swarm bu taşımayı **otomatik** yapar, uygulama çalışmaya devam eder.

7. Docker'ın Çalışma Modları

Mod	Tanım	Özellikleri
Single Mode	Docker kurulduğunda varsayılan çalışan moddur	Tek host yönetilir, Compose kullanılabilir, Swarm özellikleri kullanılamaz
Swarm Mode	Swarm aktif edildiğinde çalışan moddur	Birden fazla host yönetilir, cluster oluşturulur, servisler dağıtılır, High Availability ve Load Balancing sağlanır

8. Docker Compose ile Docker Swarm Arasındaki Fark

	Docker Compose	Docker Swarm
Yönetim kapsamı	1 bilgisayar	Host1 + Host2 + Host3 → tek cluster
Container dağılımı	Tüm container'lar aynı host'ta	Container'lar tüm host'lara dağıtılır
Host sayısı	Tek host yönetir	Birden fazla host yönetir

Docker Compose: 1 Bilgisayar → Birden fazla Container
Docker Swarm: Host1 + Host2 + Host3 → Tek Cluster → Container'lar dağıtılır

9. Swarm'ın Sağladığı Avantajlar

- Çok sayıda host'u tek sistem gibi yönetir.
- Servisleri otomatik dağıtır.
- Kaynakları ortak havuz gibi kullanır.
- Sunucu çökse bile uygulamayı çalıştırmaya devam eder.
- Load Balancing yapar.
- Ölçeklendirme (**Scaling**) sağlar.

10. Docker Swarm Tarihçesi

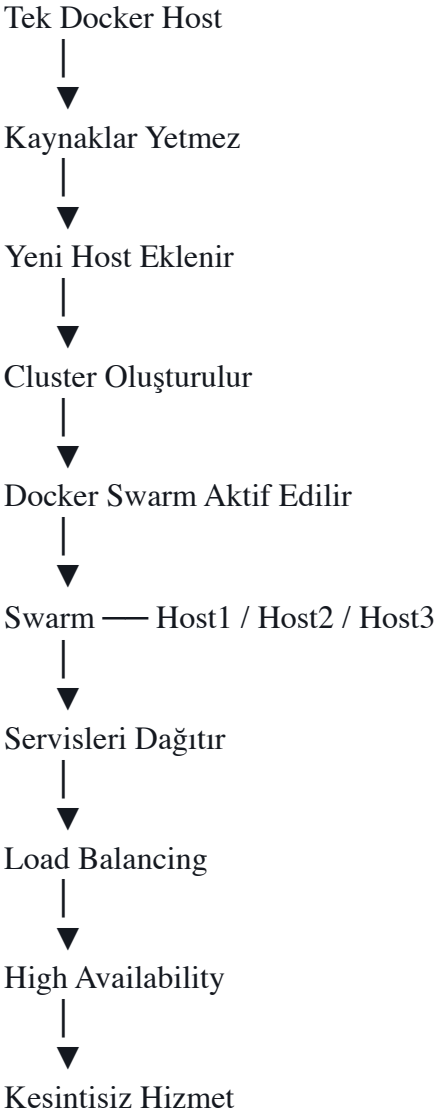
İlk sürüm (**Classic Swarm**) karmaşık kuruluyordu ve pek beğenilmedi. 2016'da Docker, **SwarmKit**'i tanıttı:

- Docker Engine içine gömülü geldi.
- Kurulumu büyük ölçüde kolaylaştı.
- Ücretsiz olarak kullanılmaya başlandı.

11. Swarm Yönetim Komutları (İleride Detaylandırılacak)

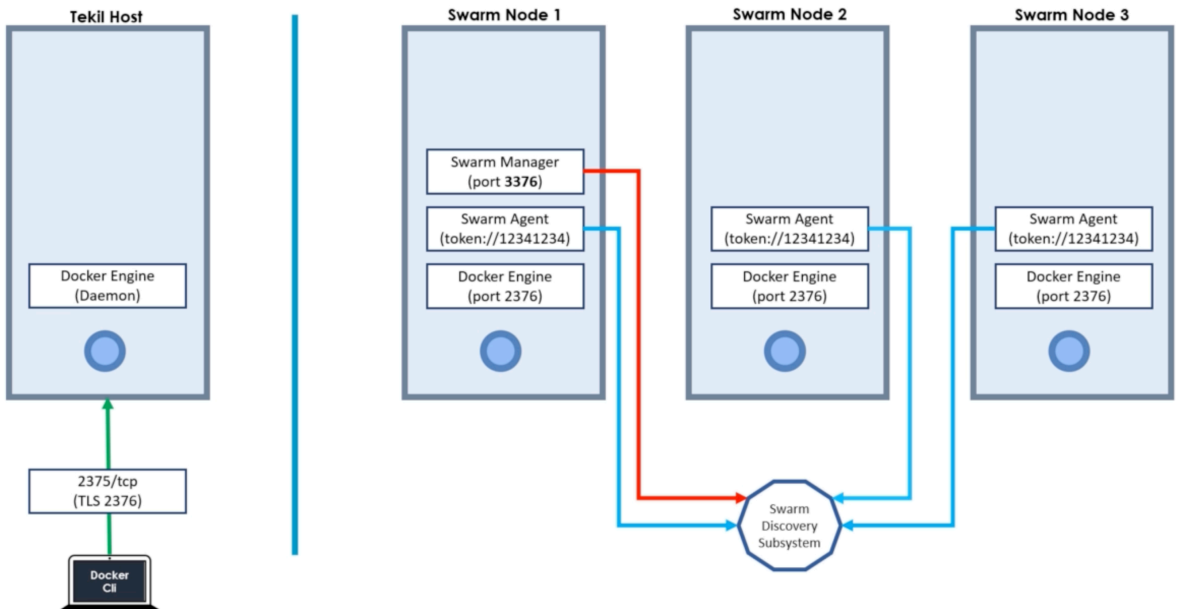
```bash
docker swarm      # Cluster oluşturma/katılma işlemleri
docker node       # Cluster'daki host'ları (node) yönetme
docker service    # Servisleri oluşturma/ölçekleme
docker stack      # Compose dosyasını Swarm üzerinde çalıştırma
```

12. Genel Akış



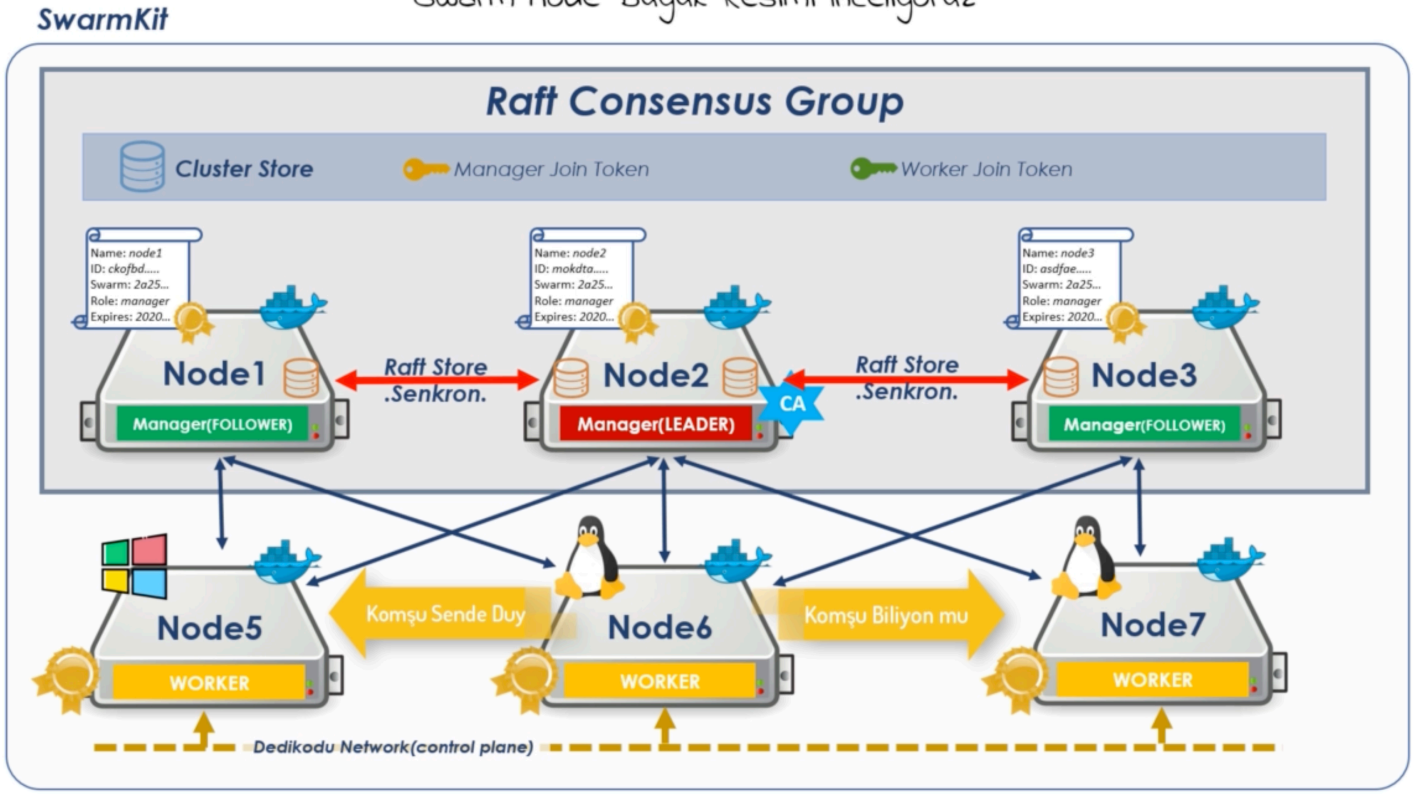
Swarm Mode Büyük Resim

Swarm Mode Büyük Resimi inceliyoruz



Swarm Mode Büyük Resim

Swarm Mode Büyük Resimi inceliyoruz



16. Bölüm: Docker Swarm — Mimari, Komutlar ve İleri Özellikler

1. Orkestratörde Olması Gereken Temel Özellikler

Bir orkestrasyon aracının (Swarm, Kubernetes vb.) sağlaması gereken özellikler:

Özellik	Tanım
High Availability	Bir sunucu kapansa bile servis başka sunucuda kesintisiz devam eder
Scaling	İhtiyaca göre container sayısını artırıp azaltabilme
Load Balancing	Gelen istekleri tüm sunuculara dengeli dağıtma
Service Discovery / Routing	Servislerin birbirini doğru isim ve ağ üzerinden bulması
Self-Healing	Çöken container'ı otomatik yeniden oluşturma
Replication	Aynı servisten birden fazla kopya çalıştırabilme
Güvenli Network	Servislerin izole ağlarda, sadece gerekenle haberleşmesi
Rolling Update	Uygulamayı durdurmadan yeni sürüme geçirme
Health Check	Container sağlık durumunu sürekli takip etme

→ Docker Swarm bu dokuz özelliği kendi içinde sağlar.

2. Piyasadaki Orkestrasyon Araçları

Swarm tek seçenek değil — **Kubernetes**, **Docker Swarm**, **Apache Mesos**, **Amazon ECS**, **Azure Container Service** en yaygınları. Günümüzde en popüler çözüm Kubernetes olsa da, orkestrasyon mantığını kavramak için Swarm öğrenmek değerlidir.

3. Node Kavramı ve Swarm Mimarisi

Tanım — Node: Swarm'a dahil edilen her Docker host'una **node** denir.

→ Artık tüm host'lar **tek bir sistem** gibi çalışır.

4. Node Rolleri: Manager ve Worker

Rol	Görevi
Manager	Cluster'ı yönetir, servis oluşturur, container'ları node'lara yerleştirir, node'ları takip eder, load balancing yapar, cluster bilgisini saklar
Worker	Sadece Manager'ın verdiği işleri çalıştırır, yönetim kararı vermez

5. Manager Node İçindeki Yapı: Leader ve Follower

Manager'ların kendi içinde iki alt rolü vardır:

- **Leader** → tek tanedir, tüm yönetim kararlarını alır (yeni node ekleme, servis oluşturma vb.)
- **Follower** → diğer manager'lar, lideri takip eder; lider kapanırsa aralarında seçim yaparak yeni lider belirlerler



→ Bu sayede yönetim durmaz.

Kaç Manager Olmalı?

- Test ortamı → 1 Manager yeterli
- Canlı ortam → 3, 5 veya 7 Manager (tek sayı — çoğunluk/majority ile karar almak için; örn. 3 manager varsa en az 2'si aynı kararı vermeli)

Raft Consensus Protocol → Manager'lar bu protokolle haberleşir, amacı: her manager'ın aynı bilgiye sahip olması, yanlış karar alınmaması, güvenli lider seçimi.

→ Worker'lar yönetim kararı vermez; sadece işleri yapar ve durumunu Manager'a bildirir. Manager, cluster'ın tüm güncel durumunu bilir.

Container fiziksel sunucudan bağımsızdır → bir node'da kaynak kalmazsa Swarm container'ı otomatik başka node'a taşır; sen hangi sunucuda çalıştığını düşünmezsin, önemli olan servisin çalışıyor olmasıdır.

6. Docker Swarm'ı Test Edebileceğin Ortamlar

Yöntem	Özet
Play with Docker	En kolay/hızlı yöntem, Docker Hub hesabıyla giriş, ~4 saat ücretsiz, hazır
Docker Machine + VirtualBox	Kendi bilgisayarında birden fazla sanal Docker host oluşturur
Cloud Platformları	AWS, Azure, Google Cloud, DigitalOcean — eğitmen özellikle DigitalOcean'ı öneriyor (kolay kurulum, uygun fiyat)
Fiziksel/Sanal Makineler	En gerçekçi yöntem — VM1: Manager, VM2/VM3: Worker gibi

Docker Machine tanımı: Aynı bilgisayarda veya bulutta birden fazla Docker Engine oluşturup yönetmeyi sağlayan araçtır.

bash
```
docker-machine create --driver virtualbox node1   # makine oluştur
docker-machine ls                                 # makineleri listele
docker-machine ssh node1                          # makineye bağlan
docker-machine stop node1                         # durdur
docker-machine start node1                        # başlat
docker-machine rm node1                           # sil
```
7. Swarm Kurulum ve Yönetim Komutları

`docker swarm init`

Swarm ortamını başlatan ilk komut → Single Mode'dan Swarm Mode'a geçer, ilk Manager (aynı zamanda Leader) oluşur, join token üretilir.

bash
```
docker swarm init
docker swarm init --advertise-addr 192.168.1.10
```
Join Token

bash
```
docker swarm join-token worker      # worker tokenı göster
docker swarm join-token manager     # manager tokenı göster
docker swarm join-token -q worker   # sadece token değeri
```
Token 3 parçadan oluşur: **Swarm ID** (hangi cluster), **Role Key** (worker mı manager mı), **Manager IP:Port**.

`docker swarm join`

bash
```
docker swarm join --token TOKEN 192.168.1.10:2377
```
`docker swarm leave` / `update`

bash
```
docker swarm leave     # node'u cluster'dan çıkarır
docker swarm update    # swarm genel ayarlarını günceller (sertifika vb.)
```
8. Node Yönetimi (`docker node`)

Komut	Görevi
`docker node ls`	Node'ları, durumlarını, rollerini listeler
`docker node promote node3`	Worker → Manager yapar
`docker node demote node3`	Manager → Worker yapar
`docker node update --availability drain node3`	Yeni görev almasın, mevcut task'ları da başka node'a taşısın (bakım modu)

`docker node update --availability active node3`	Bakım sonrası tekrar aktif eder
`docker node ps node3`	O node'daki çalışan task'leri gösterir

`availability` değerleri: **active** (normal), **pause** (yeni görev almaz, mevcut devam eder), **drain** (yeni görev almaz + mevcutlar taşınır).

⚠️ Önemli: Yönetim komutları yalnızca **Manager** node üzerinde çalışır — Worker'da `docker node ls` gibi komutlar hata verir.

9. Gerçek Kurulum Akışı (Özet)

```
docker info                    → Swarm inactive görülür
docker swarm init              → İlk Manager (Leader) oluşur
docker swarm join-token        → Token alınır
docker swarm join              → Diğer node'lar (Manager/Worker) eklenir
docker node ls                 → Node'lar kontrol edilir
docker node promote node3      → Worker → Manager
docker swarm leave (worker'da) → Node ayrılır, Manager'da "Down" görünür
docker node rm NODE_ID         → Down node tamamen silinir
```

10. Service, Task, Container Mantığı

Swarm öncesi: container'ı biz oluşturur, hangi host'ta çalışacağını biz belirler, çökerse biz müdahale ederiz (`docker run nginx`).

Swarm sonrası: artık container değil, **service** yönetiriz — "Web servisi oluştur" deriz, gerisini Swarm halleder.

Tanım — Service: Swarm üzerinde çalıştırılmak istenen uygulamanın tanımıdır (servis adı, image, port, network, volume, CPU/RAM limiti, replica sayısı içerir).

Tanım — Task: Swarm'ın bir container'ı oluşturma, çalıştırma ve **takip etme görevidir**. Task, container'ın işi değil, Swarm'ın o container üzerindeki yönetim görevidir.

```
1 Task = 1 Container
```

İlişki:

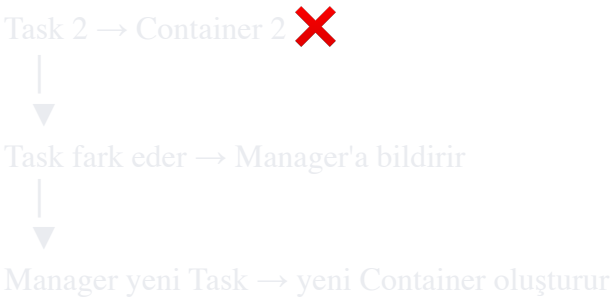
bash

```
docker service create --name web --replicas 3 nginx
```

```
Web Service
 ├── Task 1 → Container 1
 ├── Task 2 → Container 2
 └── Task 3 → Container 3
```

→ 1 Service, 3 Task, 3 Container.

Self-Healing akışı:



→ Replica sayısı korunur.

11. Service Türleri

Tür	Açıklama
Replicated (varsayılan)	Kaç kopya olacağını sen belirlersin (`--replicas 3`) → belirtilen sayıda container oluşur
Global	Replica yazılmaz, Swarm her node'a otomatik 1 container koyar → node sayısı artarsa container sayısı da artar

12. Servis Yönetim Komutları

bash

```
# Servis oluşturma
docker service create --name web-demo -p 8080:80 nginx
docker service create --name web --replicas 4 -p 8080:80 nginx
docker service create --mode global nginx

# Listeleme ve inceleme
docker service ls                  # servisleri listele (REPLICAS: 1/1 gibi)
docker service ps web-demo         # task'ların hangi node'da çalıştığını gör
docker service inspect web --pretty  # servis detayları

# Ölçeklendirme
docker service scale web-demo=6
docker service update --replicas 10 web-demo

# Belirli node'da çalıştırma (constraint)
docker service create --constraint node.hostname==node1 mysql
docker service create --constraint node.hostname!=node1 mysql
docker service create --constraint node.role==manager mysql
docker service create --constraint node.role==worker mysql
```

→ `constraint` özellikle veritabanı gibi belirli sunucuda çalışması gereken servislerde kullanılır.

Uygulamada gözlemlenen Self-Healing örneği:

13. Monitoring ve Yönetim Araçları

Araç	Ne İşe Yarar
Docker Swarm	Cluster'ı web arayüzünde görsel gösterir — hangi servis hangi node'da, task çalışıyor mu (yeşil) durmuş mu (kırmızı)
Portainer	Container/Image/Volume/Network/Service/Stack'i terminal olmadan tamamen görsel arayüzden yönetir, kendi Cluster Visualizer'ı da vardır
Sematext	Servisleri değil, **sunucu/Docker performansını** izler (CPU, RAM, disk, network

14. Overlay Network

Tanım: Farklı node'lardaki container'ların **aynı ağdaymış gibi güvenli** haberleşmesini sağlayan network türüdür.

- `bridge` → tek host için
- `overlay` → birden fazla host (Swarm) için

bash
```
docker network create --driver overlay ortak-network
```
→ Servisler birbirini **IP yerine servis adıyla** bulur (örn. WordPress'te `DB_HOST=mysql` yazmak yeterli). Uygulamada MySQL, WordPress ve phpMyAdmin aynı overlay network'e bağlanarak sorunsuz haberleşti.

En büyük avantajı: Overlay Network olmadan sadece servisin çalıştığı node'un IP'sinden erişilebilirken, Overlay Network sonrası **her node'un IP'sinden** aynı servise ulaşılabilir hale gelir.

15. Rolling Update

Tanım: Çalışan bir servisi **durdurmadan**, eski container'ları sırayla yenisiyle değiştirerek yeni sürüme geçme işlemidir.

bash
```
docker service update \
  --image myapp:v2 \
  --update-parallelism 2 \
```

--update-delay 10s \
web-demo

- `--update-parallelism 2` → aynı anda 2 task güncellenir
- `--update-delay 10s` → güncellemeler arası 10 saniye beklenir

→ Uygulama, kullanıcı hiçbir kesinti hissetmeden v1'den v2'ye geçer.

16. Routing Mesh

Tanım: Docker Swarm'ın tüm node'ları **tek bir giriş noktası** gibi göstermesini sağlayan yerleşik load balancing mekanizmasıdır.

→ Servis hangi node'da çalışırsa çalışsın, cluster'daki **her node'un IP'si** üzerinden erişilebilir.

İstek → Node1:8080 (container yok)
|
▼
Routing Mesh → "Task hangi node'da?" → Node2 → Container → cevap döner

Bunu sağlayan altyapı:

- **Ingress Overlay Network** → Swarm kurulunca otomatik oluşur, tüm node'lar buna dahildir.
- **VIP (Virtual IP)** → her servise Swarm tarafından atanan sanal IP; Routing Mesh isteği önce VIP'ye, oradan uygun task/container'a yönlendirir.

bash
docker service create --name web --replicas 2 -p 8080:80 nginx
→ Container sadece Node2 ve Node5'te olsa bile, Node1/Node3/Node4 IP'lerinden de aynı servise erişilebilir.

Bu Bölümde Öğrendiklerimiz

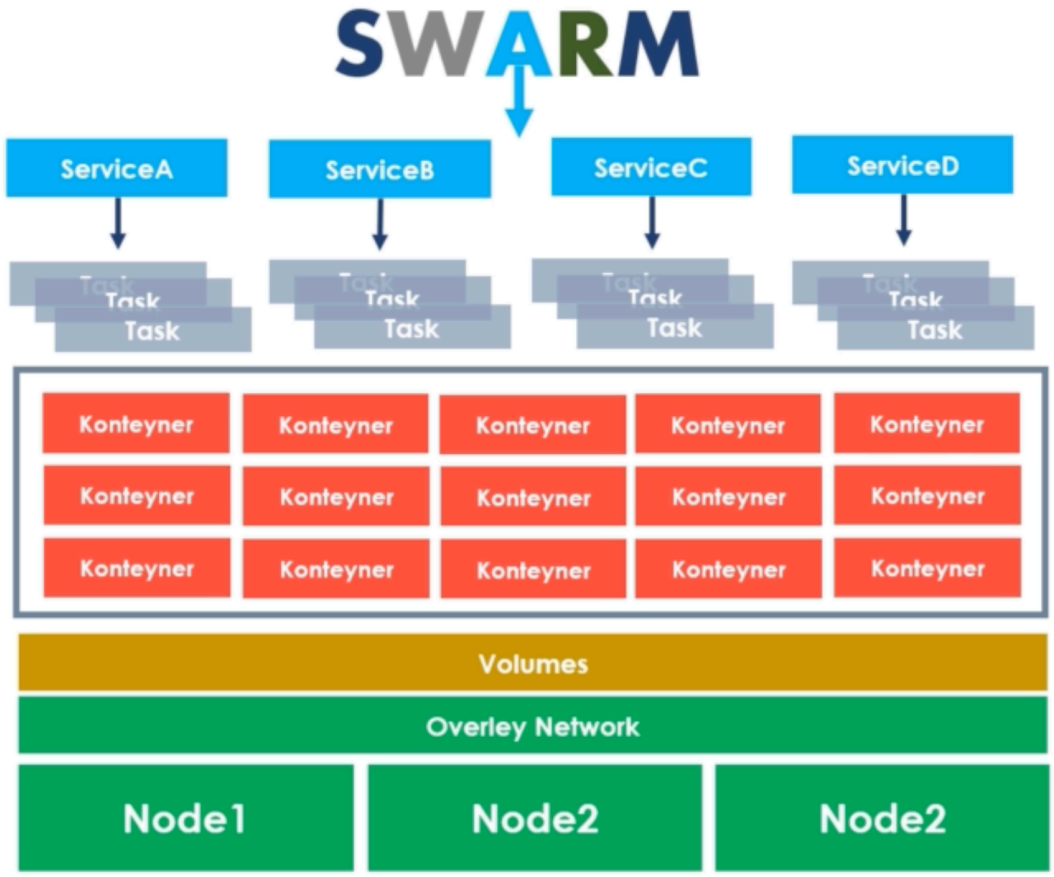
- **Routing Mesh**, Swarm'ın yerleşik **Load Balancer** özelliğidir.
- Servis hangi node'da çalışırsa çalışsın, **cluster'daki tüm node IP'leri üzerinden erişilebilir.**
- Routing Mesh bu işlemi **Ingress Overlay Network** üzerinden gerçekleştirir.
- Her servis için bir **VIP (Virtual IP)** oluşturulur ve gelen istekler uygun task/container'a yönlendirilir.
- Kullanıcının isteği yanlış node'a gelse bile Swarm onu doğru node'daki container'a otomatik olarak iletir.

Ezberlik Cümle

Bu bölümde, Docker Swarm'da servislerin tüm node'lar üzerinden erişilebilir olmasını sağlayan Routing Mesh mekanizmasını, bunun Ingress Overlay Network ve yerleşik Load Balancer ile nasıl çalıştığını, gelen isteklerin uygun task/container'a otomatik yönlendirilmesini ve böylece hangi node'da çalışırsa çalışsın servislerin kesintisiz olarak erişilebilir kalmasını öğrendik.

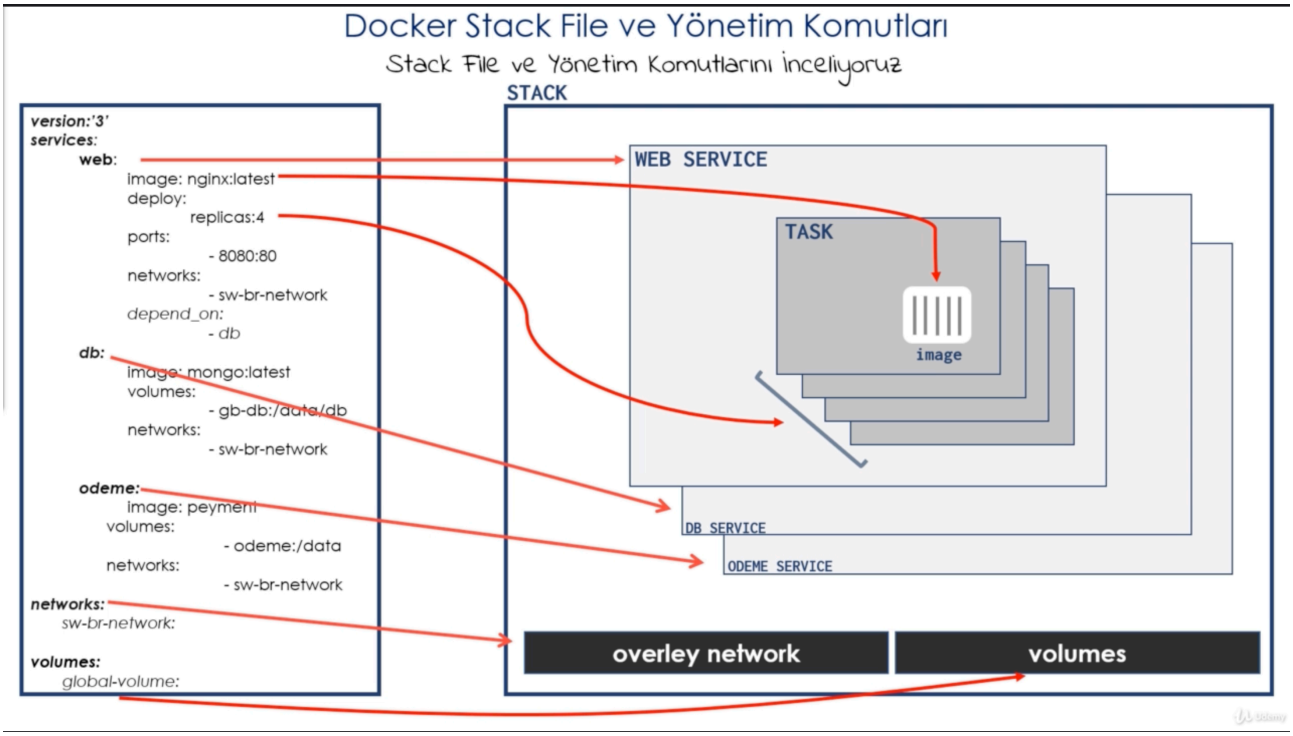
Docker Stack Nedir ?

Docker Swarm Stack Nedir öğreniyoruz



Docker Stack File ve Yönetim Komutları

Stack File ve Yönetim Komutlarını inceliyoruz



16. Bölüm: Docker Stack, Docker Secret ve Kubernetes'e Giriş

1. Docker Stack Nedir?

Tanım: Docker Stack, Docker Swarm üzerinde çalışan birden fazla servisi, network'ü ve volume'ü **tek bir docker-compose.yml dosyası** ile oluşturup yönetmemizi sağlayan yapıdır.

Swarm'da her servisi (`web`, `database`, `redis`, `worker`, network, volume) tek tek `docker service create` ile yazmak yerine, Stack sayesinde hepsi tek dosyada tanımlanır ve tek komutla ayağa kaldırılır.

Docker Compose ile Docker Stack Arasındaki İlişki

	Docker Compose	Docker Stack
Çalışma ortamı	Tek Docker host	Docker Swarm (cluster)
Kullandığı dosya	docker-compose.yml	Aynı docker-compose.yml (Stack dosyası)

→ Docker Stack, **aynı Compose dosyasını Swarm ortamında** çalıştıran yapıdır.

Geliştirme akışı:

→ Aynı dosya geliştirme, test ve production'da kullanılabilir.

2. Docker Stack Dosyası ve `deploy` Bölümü

Docker Stack dosyası aslında bildiğimiz `docker-compose.yml` dosyasıdır; farkı **Swarm'a özel `deploy` bölümünü** de desteklemesidir.

Deploy Anahtarı	Görevi
`replicas`	Kaç kopya (task/container) çalışacağını belirler → `deploy: replicas: 4`
`mode`	`replicated` (belirlenen sayıda kopya) veya `global` (her node'da 1 kopya)

`labels`	Servise etiket ekler
`placement`	Servisin hangi node'da çalışacağını belirler (örn. sadece Manager, sadece Worker, belirli OS)
`resources`	CPU/RAM limitlerini belirler (maksimum kullanım, ayrılacak minimum kaynak)

Docker Stack Nasıl Çalışır?

Uygulamalı Örnek (Voting App)

İnternetten hazır bir Docker Stack (Voting App) projesi indirildi, `docker-stack.yml` içindeki `deploy`, `replicas`, `placement`, `update_config` ayarları incelendi:

bash

```
docker stack deploy -c docker-stack.yml proje    # stack'i ayağa kaldır
docker stack ls                                  # çalışan stack'leri listele
docker stack services proje                      # stack içindeki servisleri listele
docker stack ps proje                            # task'ların hangi node'da çalıştığını gör
```

Test edilen senaryolar:

- WordPress servisi için `placement` kuralı → "sadece Worker Node'larda çalışsın" → gerçekten sadece Worker'larda oluştuğu doğrulandı.
- Result servisinin replica sayısı artırıldı → task'ların node'lara yeniden dağıldığı gözlemlendi.
- Bir node **Drain** moduna alındı → üzerindeki task'lar otomatik diğer node'lara taşındı, sonra node tekrar **Active** yapılarak cluster'a döndü.
- Visualizer ile servis/task dağılımı görsel olarak izlendi.

→ Tek bir `docker-stack.yml` dosyasıyla network, volume, servis, replica, placement, resource limitleri ve rolling update ayarları aynı anda oluşturulup Swarm ortamı tamamen ayağa kaldırılabiliyor.

3. Docker Secret Nedir?

Tanım: Docker Secret, şifre, kullanıcı adı, token, SSL sertifikası, SSH private key gibi gizli bilgileri **güvenli şekilde saklamak ve container'lara iletmek** için kullanılan bir Swarm özelliğidir.

Problem: Eskiden şifreler doğrudan yazılıyordu:

yaml
```yaml
environment:
  MYSQL_ROOT_PASSWORD: 123456
```
→ Dosyayı gören herkes şifreyi görebilir, güvenli değildir.

Docker Secret Nasıl Çalışır?

Servis silinirse → container, task ve memory'deki Secret otomatik silinir; Secret asla açık halde sistemde kalmaz.

Secret Yönetim Komutları

bash
```bash
docker secret create db_user db_user.txt          # dosyadan secret oluştur
echo "123456" | docker secret create db_password -  # komut satırından oluştur (- işareti)
docker secret ls                                  # secret'ları listele
docker secret inspect db_user                     # detayları göster (içeriği DEĞİL)
docker secret rm db_user                          # secret sil
```
Servise Secret Ekleme/Kaldırma/Güncelleme

bash
```bash
docker service create --secret db_password mysql        # servis oluştururken ekle
docker service update --secret-add db_password web      # çalışan servise sonradan ekle
docker service update --secret-rm db_user web           # servisten kaldır
docker service update --secret-rm old_secret --secret-add new_secret web   # güncelle
```
Uygulama, şifreyi environment'a yazmak yerine dosya yolu üzerinden okur:

yaml
```
environment:
  POSTGRES_USER_FILE: /run/secrets/db_user
  POSTGRES_PASSWORD_FILE: /run/secrets/db_password
```
Docker Compose/Stack İçinde Secret Tanımlama

yaml
```
secrets:
  db_user:
    file: ./db_user.txt
  db_password:
    file: ./db_password.txt

services:
  db:
    secrets:
      - db_user
      - db_password
    environment:
      MYSQL_USER_FILE: /run/secrets/db_user
      MYSQL_PASSWORD_FILE: /run/secrets/db_password
```
⚠️ **Dikkat:** Compose/Stack çalıştırıldıktan sonra `db_user.txt` ve `db_password.txt` gibi düz metin dosyaları **silinmelidir** — diskte açık kalırsa güvenlik riski oluşturur.

Docker Secret'ın avantajları:

- Şifreler Dockerfile'da veya Compose/Stack dosyasında düz metin görünmez
- Şifrelenmiş şekilde saklanır
- Sadece ihtiyacı olan container erişebilir
- Servis silinince Secret da otomatik kalkar

4. Uygulamalı Örnek: Voting App (Overlay Network + Servisler)

Bu örnekte önce **frontend** ve **backend** olmak üzere iki Overlay Network oluşturuldu. Ardından `docker service create` komutlarıyla **Vote, Redis, DB (PostgreSQL), Worker ve Result** servisleri oluşturuldu; her servis ihtiyacına göre ilgili network'e bağlandı, bazı servislere replica, volume ve port yönlendirmesi tanımlandı.

Kontrol adımları:

bash
```
docker service ls              # servislerin durumu (replica, image, port)
docker service ps <servis>     # task'ların hangi node'da çalıştığı
docker service logs <servis>   # servis logları → Redis ve DB bağlantısının başarılı olduğu doğrulandı
```
Son olarak tarayıcıdan uygulama test edildi — oy verme ve sonuç ekranlarının sorunsuz çalıştığı görüldü.

Bu örnekten çıkarılacak sonuçlar:

- Overlay Network sayesinde farklı node'lardaki servisler birbirleriyle **servis adıyla** haberleşebilir.
- `docker service create` ile Swarm üzerinde servis oluşturulur.
- `docker service ls / ps / logs` ile servisler izlenir ve yönetilir.
- Swarm, servisleri farklı node'lara dağıtarak **çok servisli** bir uygulamanın tek bir cluster üzerinde sorunsuz çalışmasını sağlar.

5. Kubernetes'e Giriş

Uygulama büyüdükçe (web + veritabanı + Redis + API gibi birçok container birlikte) bunları tek tek yönetmek zorlaşır. **Kubernetes**, bu container'ları otomatik yöneten bir **container orkestrasyon platformudur**.

Tanım: Kubernetes, container tabanlı uygulamaların **deployment (dağıtım)**, **scaling (ölçeklendirme)**, **load balancing (yük dengeleme)**, **rolling update (güncelleme)** ve genel yönetimini otomatikleştiren açık kaynaklı bir orkestrasyon platformudur.

Önemli bilgiler:

- Google tarafından geliştirilmiştir, **Go** dili ile yazılmıştır.
- 2014'te açık kaynak olarak yayınlandı.
- Günümüzde **CNCF (Cloud Native Computing Foundation)** tarafından geliştiriliyor.
- Fiziksel sunucu, sanal makine, AWS/Azure/GCP gibi bulut platformlarında çalışabilir.

Swarm Varken Neden Kubernetes?

Swarm birçok işi yapabilse de Kubernetes:

- Daha gelişmiş otomatik yönetim sunar
- Daha güçlü ölçeklendirme yapar
- Çöken container'ları daha başarılı yönetir
- Büyük/karmaşık sistemlerde daha esnektir
- Cloud ortamlarıyla daha güçlü entegre çalışır

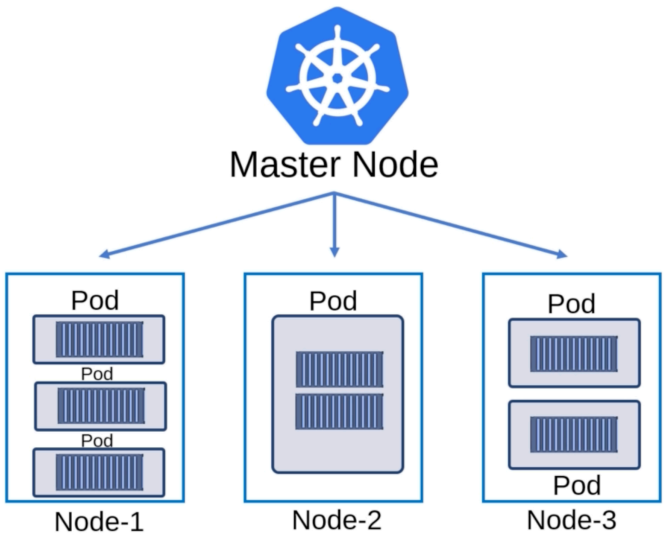
→ Bu yüzden günümüzde büyük ölçekli projelerde en yaygın kullanılan orkestrasyon platformu **Kubernetes**'tir.

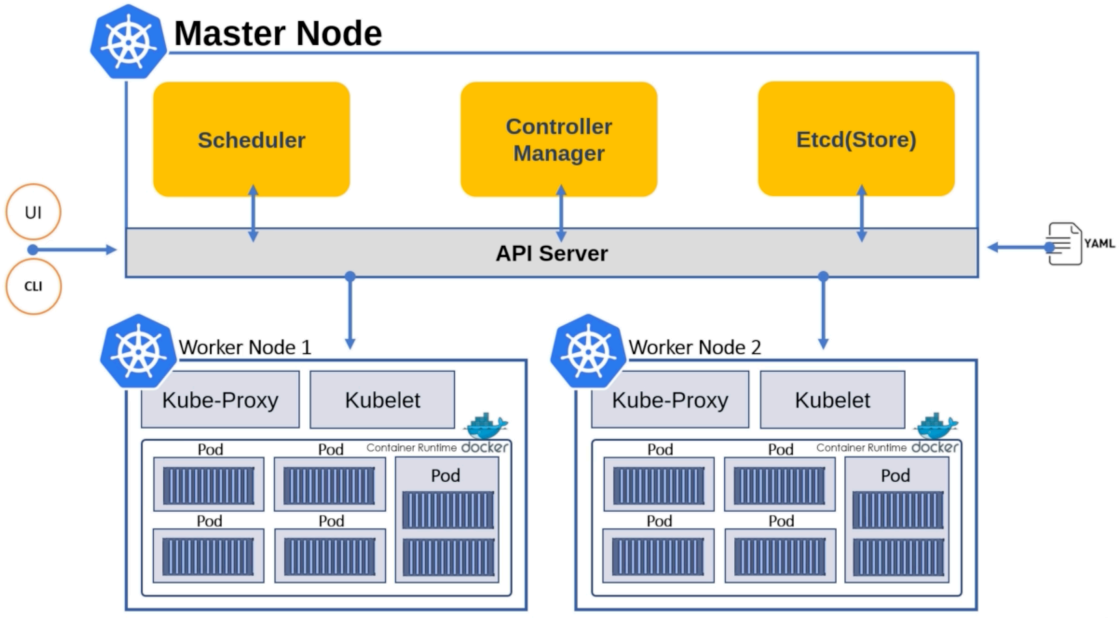
Bu bölümde öğrendiklerimiz

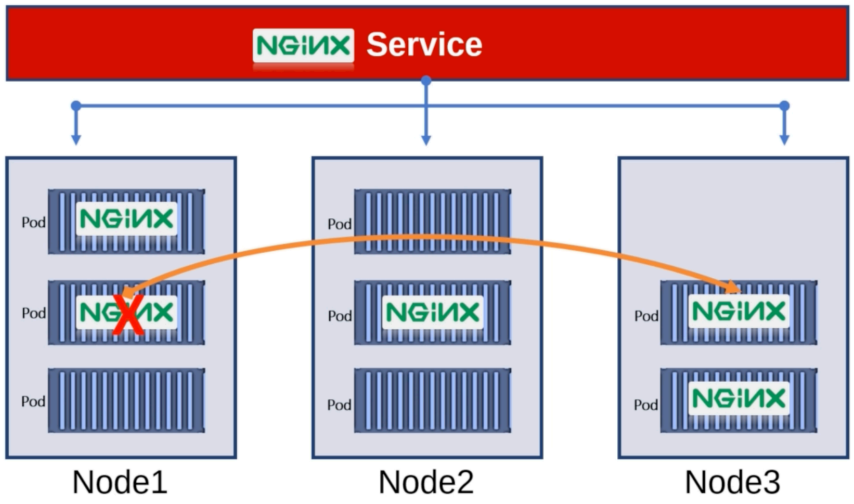
- Kubernetes'in neden ortaya çıktığını,
- Kubernetes'in bir **container orkestrasyon platformu** olduğunu,
- Deploy, Auto Scaling, Load Balancing, Self-Healing, Rolling Update ve Storage yönetimini otomatik yaptığını,
- Google tarafından geliştirildiğini ve CNCF tarafından desteklendiğini,
- Docker Swarm'a göre daha güçlü ve esnek olduğu için yaygın olarak tercih edildiğini öğrendik.

Kısaca Ezberle

Kubernetes (K8s), Docker container'larını tek tek yönetmek yerine; deploy, auto scaling, load balancing, self-healing, rolling update ve network/depolama işlemlerini otomatik gerçekleştiren, Google tarafından geliştirilen açık kaynaklı bir container orkestrasyon platformudur.


Master Node
Pod
Pod
Pod
Pod
Pod
Pod
Node-1
Node-2
Node-3


Master Node
Scheduler
Controller Manager
Etcd(Store)
UI
CLI
API Server
YAML
Worker Node 1
Worker Node 2
Kube-Proxy
Kubelet
Container Runtime
docker
Pod


NGINX Service
Pod
NGINX
Node1
Node2
Node3

17. Bölüm: Kubernetes Mimarisi ve Lokal Kurulum

1. Kubernetes Nedir? (Tanım — Tekrar)

Tanım: Kubernetes, container tabanlı uygulamaların **dağıtılmasını (deployment)**, **ölçeklendirilmesini (scaling)**, **yük dengelemesini (load balancing)**, **güncellenmesini (rolling update)** ve genel yönetimini otomatikleştiren, açık kaynaklı bir **container orkestrasyon platformudur**.

Docker ile uygulamaları container içinde çalıştırabiliyoruz, ama uygulama büyüdükçe (web + veritabanı + Redis + API gibi birçok container birlikte) sadece container oluşturmak yetmiyor — bunları tek tek yönetmek zorlaşıyor. İşte bu noktada Kubernetes devreye girer ve tüm container'ları **otomatik** yönetir.

Kubernetes'in otomatik yaptığı işlemler:

- Container'ları çalıştırmak (**Deploy**)
- Trafik arttığında yeni container oluşturmak (**Auto Scaling**)
- Gelen istekleri container'lara dengeli dağıtmak (**Load Balancing**)
- Çöken container'ları yeniden başlatmak (**Self-Healing**)
- Uygulamaları kesintisiz güncellemek (**Rolling Update**)
- Network ve depolamayı yönetmek (**Networking & Storage**)

Önemli bilgiler:

- Google tarafından geliştirilmiştir, **Go** dili ile yazılmıştır.
- 2014'te açık kaynak olarak yayınlanmıştır.
- Günümüzde **CNCF (Cloud Native Computing Foundation)** tarafından geliştirilir.
- Fiziksel sunucu, sanal makine, AWS/Azure/GCP gibi bulut platformlarında çalışabilir.

2. Genel Mimari: Master ve Worker Node

Kubernetes de Swarm gibi **iki ana bölümden** oluşur:

- **Master (Control Plane)** → sistemin **beynidir**, tüm kararları alır.
- **Worker Node** → uygulamaların (container'ların) gerçekten çalıştığı yerdir.

3. Worker Node Bileşenleri

Container Runtime

Docker (veya başka bir container runtime) burada çalışır. Görevi: image indirmek, container başlatmak, container durdurmak.

Pod

Tanım: Kubernetes'in **en küçük çalışma birimidir**.

→ Container Kubernetes'te **doğrudan çalışmaz**, mutlaka bir **Pod** içinde çalışır. Genellikle 1 Pod = 1 Container şeklinde kullanılır.

Kubelet

Tanım: Worker Node'un **ajanıdır**. Görevi: Master'dan gelen emirleri almak, Pod'u oluşturmak, çalışma durumunu sürekli Master'a bildirmek.

→ Ezber: **Kubelet = Worker ile Master arasındaki haberci.**

Kube Proxy

Tanım: Network yöneticisidir. Görevi: Pod IP'lerini yönetmek, Load Balancing yapmak, ağ trafiğini yönlendirmek.

4. Master (Control Plane) Bileşenleri

Master, karar veren taraftır.

API Server

Tanım: Kubernetes'in **giriş kapısıdır**. Bütün istekler önce buraya gelir.

etcd

Tanım: Kubernetes'in **veritabanıdır**. Cluster bilgileri, Pod bilgileri, Node bilgileri ve ayarlar burada saklanır.

→ Ezber: **etcd = Kubernetes'in hafızasıdır.**

Scheduler

Tanım: Yeni oluşturulacak Pod'un **hangi Worker Node üzerinde** çalışacağına karar verir.

Controller Manager

Tanım: Sistemi sürekli kontrol eder, istenen Pod sayısı ile şu anki durumu karşılaştırır.

→ Sistemi **istenen durumda (Desired State)** tutar.

5. Bir Pod Oluşturulurken Ne Oluyor? (Uçtan Uca Akış)

6. Swarm ile Karşılaştırma

En önemli fark:

Swarm	Kubernetes
Doğrudan **container** yönetilir	Container değil, **Pod** yönetilir

7. Kubernetes'i Lokal Kurma

Yöntem 1: Minikube

Bilgisayarda küçük bir Kubernetes cluster'ı oluşturur; Windows, macOS ve Linux'ta çalışır. Öğrenmek/test etmek için en yaygın araçlardan biridir.

Yöntem 2: Docker Desktop

Docker Desktop içinde hazır Kubernetes desteği bulunur:

```
Docker Desktop → Settings → Kubernetes → Enable Kubernetes
```

Bir süre sonra **"Kubernetes Running"** olur ve lokal cluster hazır hale gelir.

8. kubectl Komutu

Docker'da yönetim komutu `docker` iken, Kubernetes'te bu görevi **`kubectl`** üstlenir.

Cluster'ı Kontrol Etme

```
bash
kubectl version         # sürüm bilgisi
kubectl get nodes       # sistemdeki node'ları listele
```

Docker Desktop kullanıldığında tek node görünür:

```
NAME             ROLE
docker-desktop   master
```

İlk Deployment Oluşturma

```
bash
kubectl create deployment myweb --image=nginx
```

Bu komut: **Deployment** oluşturur → Deployment bir **Pod** oluşturur → Pod içinde Nginx container'ı çalışır.

```
Deployment → Pod → Container (Nginx)
```

Pod ve Deployment Listeleme

```
bash
kubectl get pods            # çalışan pod'ları listeler (örn. myweb-xxxxx)
kubectl get deployments     # deployment adı, kaç pod çalışıyor, kaçı hazır
```

Pod isimlerinin sonundaki rastgele karakterler Kubernetes tarafından otomatik eklenir.

Scale İşlemi

bash
```
kubectl scale deployment myweb --replicas=4
```

```
Deployment
   |
   ├──────┬──────┬──────┐
Pod1 Pod2 Pod3 Pod4
```

→ Kubernetes otomatik olarak 4 Pod oluşturur.

Pod İçine Girme

Docker'daki `docker exec -it` komutunun Kubernetes karşılığı:

bash
```
kubectl exec -it POD_ADI -- sh
```

Uygulamayı Dışarı Açma

Pod'lar yalnızca cluster içinde çalışır; tarayıcıdan erişmek için servis oluşturulur:

bash
```
kubectl expose deployment myweb
kubectl get services      # servis ve port bilgilerini gösterir
```

```
Deployment → Pod → Service → Browser
```

Replica, bir uygulamanın aynı anda çalışan birden fazla kopyasını ifade eder; Docker Swarm'da bu kopyalar container olarak, Kubernetes'te ise Pod olarak oluşturulur ve amaç hem artan kullanıcı yükünü paylaşmak (Load Balancing) hem de bir kopya çökerse diğerlerinin çalışmaya devam etmesini sağlayarak kesintisiz hizmet (High Availability) sunmaktır.

DOCKER YOLCULUĞU

1. Geleneksel Mimari

Application
|
Operating System
|
Hardware

Donanım verimsiz kullanılıyor.

↓

2. Sanallaştırma (Virtual Machine)

Application
|

Guest OS
|
Virtual Machine
|
Hypervisor
|
Hardware

Aynı sunucuda birden fazla işletim sistemi çalıştırılabilir.
Ancak fazla RAM ve CPU tüketir.

↓

3. Container Teknolojisi

Application
|
Container
|
Docker Engine
|
Host Operating System
|
Hardware

Hafif
Hızlı
Taşınabilir
Her ortamda aynı şekilde çalışır.

↓

4. Docker'ın Temel Yapıları

Docker Image
|
docker run
|
Container

Container Yönetimi
- **docker run**
- **docker exec**
- **docker logs**
- **docker inspect**
- **docker stats**

↓

5. Kalıcılık ve Haberleşme

Volume
│
├── Named Volume
└── Bind Mount

Network
│
├── Bridge
├── Host
├── None
└── Overlay

↓

6. Dockerfile

Kod
│
Dockerfile
│
docker build
│
Docker Image
│
docker run
│
Container

Dockerfile Komutları
FROM
RUN
COPY
ADD
WORKDIR
CMD
ENTRYPOINT

↓

7. Docker Compose

Birden Fazla Servis

docker-compose.yml

│
├── Web
├── Backend
├── Database

├── Redis
├── Volume
└── Network

|
docker compose up

▼

Tüm sistemi tek komutla ayağa kaldırır.

↓

8. Docker Swarm

Tek Sunucu
|
Yetersiz Kalır
|
Yeni Sunucular
|
Cluster
|
Swarm

Swarm;

• Service oluşturur
• Replica yönetir
• Load Balancing yapar
• Self-Healing sağlar
• High Availability sağlar

↓

9. Docker Stack

---

docker-compose.yml
|
docker stack deploy
|
Swarm Cluster

Compose dosyası artık
production ortamında çalışır.

↓

10. Docker Secret

Şifreler
API Key
Token
SSL

|

Docker Secret

|

/run/secrets

Güvenli saklanır.
Sadece ilgili servis erişebilir.

↓

11. Kubernetes

Control Plane (Master)
|
API Server
Scheduler
Controller Manager
etcd
|
——————————
Worker Node
|
Kubelet
Kube Proxy
Container Runtime
|
Pod
|
Container

Kubernetes;

Deployment
Auto Scaling
Load Balancing
Self-Healing
Rolling Update
Storage
Networking

işlemlerini otomatik yönetir.